

Figure 3.18 Responding to a CORS request using the Access-Control-Allow-Origin header

the conversation by saying "Yup, that client can have access." If this header isn't present, the CORS request will fail.

The Access-Control-Allow-Origin header is an additional response header layered onto the response. It shouldn't affect any other response parameters. If the resource can't be found and returns a 404 error, it should continue returning a 404, even with the Access-Control-Allow-Origin header. Figure 3.18 shows how a server can use the Access-Control-Allow-Origin header to respond to a CORS request.

The value of the Access-Control-Allow-Origin header can be either a wildcard or an origin value. The wildcard value says that clients from any origin can access the resource, while the origin value only gives access to a specific client. Here is an example of both header values.

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Origin: http://localhost:1111
```

Let's look at how to use these header values.

3.5.2 Access-Control-Allow-Origin with a wildcard (*) value

An Access-Control-Allow-Origin header with the value `*` indicates that *any* client can access this resource. In fact, the simplest way to add CORS support to a server is to add `Access-Control-Allow-Origin: *` to every response. Let's modify the sample to do just that.

Listing 3.4 introduces a new piece of middleware to the server named `handleCors`. All CORS-related functionality will go in this middleware. The `handleCors` function adds an Access-Control-Allow-Origin header to the response, then calls `next()` to continue processing the request. (Calling `next()` is a standard pattern that all Express middleware components must follow to continue processing the request.) Finally, you attach the `handleCors` middleware to the server processing pipeline.

When you restart the server and reload the client at http://localhost:1111/client.html, you should see the blog posts loaded on the page. If you examine the request in the Network tab, you'll now see both the HTTP request and response, with the Origin header in the request and the Access-Control-Allow-Origin header in the response, as shown in figure 3.19. Congratulations—with that one line of code, you've added CORS support to the server!